

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación



[CC3069] Computación Paralela y Distribuida

Proyecto 1

Nombre	Carné
Daniel Chet	231177
Dulce Ambrosio	231143
Cristian Túnchez	231359



Índice

1. Introducción	4
2. Antecedentes	4
2.1 Memoria compartida y OpenMP	4
2.2 Speedup, eficiencia y límites	4
2.3 Método PCAM	4
3. Diseño de la solución	5
3.1 Requisitos funcionales	5
3.2 Arquitectura modular	5
3.3 Generación, física y colisiones	5
3.4 Gestión de recursos	5
4. Paralelización con PCAM	5
4.1 Partitioning	5
4.2 Communication	6
4.3 Agglomeration y Mapping	6
5. Metodología experimental	6
5.1 Benchmark físico	6
5.2 Campaña FPS	6
5.3 Entorno y reproducibilidad	6
6. Resultados	7
6.1 Tiempo, speedup y eficiencia	7
6.2 Rendimiento visual	7
7. Análisis y discusión	9
7.1 Tamaño del problema	9
7.2 Escalabilidad y eficiencia	9
7.3 FPS y experiencia	9
7.4 Selección práctica	9
8. Validación de corrección	10
9. Limitaciones	10
10. Conclusiones	10
11. Recomendaciones	11
Referencias	11
Anexos	12

Anexo 1 - Diagrama de flujo	12
Anexo 2 - Catálogo de funciones y tipos	16
Anexo 3 - Bitácora de pruebas	22

1. Introducción

El proyecto desarrolla un screensaver parametrizado por N burbujas. Cada elemento tiene posición, velocidad, radio y color RGBA; se mueve dentro de un canvas OpenGL de 800 x 600, rebota en los bordes y colisiona con otras burbujas. El título de la ventana informa los cuadros por segundo. Esta combinación aporta un problema visual verificable y un kernel de física que puede descomponerse entre hilos.

El objetivo fue construir una referencia secuencial correcta y transformar las partes independientes a OpenMP sin alterar el resultado. La evaluación separa dos preguntas: cuánto se acelera el kernel de física y qué experiencia produce el programa completo. Por ello el speedup se calcula con un benchmark sin renderizado, mientras los FPS se presentan como evidencia suplementaria.

El informe describe la arquitectura modular actual, aplica PCAM, documenta los mecanismos de sincronización, presenta diez mediciones por configuración y conserva los CSV primarios para auditoría. La única parte excluida en esta etapa es el enlace e historial del repositorio, que el equipo incorporará posteriormente.

2. Antecedentes

2.1 Memoria compartida y OpenMP

En memoria compartida, varios hilos acceden al mismo espacio de direcciones. Esto reduce el costo de comunicar estructuras grandes, pero exige distinguir datos privados de datos compartidos y ordenar las fases que tienen dependencias. OpenMP ofrece directivas para crear equipos de hilos, repartir iteraciones y sincronizar accesos. La especificación define, entre otros, parallel, for, critical, barrier y single.

Una barrera establece que ningún hilo avance hasta que todos alcancen el mismo punto. Critical permite que una sola hebra ejecute una región a la vez. Single asigna una operación a una hebra e incluye una barrera implícita salvo que se use nowait. Aquí esos mecanismos protegen la combinación de pares y separan detección de resolución.

2.2 Speedup, eficiencia y límites

Para p hilos, el speedup es $S(p) = \frac{T(1)}{T(p)}$ y la eficiencia es $E(p) = \frac{S(p)}{p}$. Se usa la media de diez tiempos por configuración. Un speedup menor que uno representa desaceleración; no se oculta ni se reemplaza por una razón de FPS. La ley de Amdahl muestra que la parte serial limita la aceleración máxima incluso cuando crece p . En este caso, ordenar y resolver colisiones continúa en single.

2.3 Método PCAM

PCAM organiza el diseño en Partitioning, Communication, Agglomeration y Mapping. Primero identifica tareas y datos, luego estudia sus dependencias, después agrupa trabajo para amortizar costos y por último asigna las tareas a procesadores evitando agregar directivas sin un modelo de dependencias.

3. Diseño de la solución

3.1 Requisitos funcionales

- Recibir N por argumentos y validar rangos, texto, overflow y combinaciones de opciones.
- Mostrar varios colores, movimiento y física en un canvas no menor que 640 x 480.
- Desplegar FPS y conservar versiones secuencial y OpenMP funcionales.
- Medir tiempo, speedup y eficiencia con por lo menos diez repeticiones.
- Liberar ventana, buffers, shaders y textura aun cuando falle la inicialización.

3.2 Arquitectura modular

main.cpp solo interpreta el resultado de la CLI y despacha un modo. “*generation.cpp*” crea un estado determinista y calcula checksums. “*physics.cpp*” contiene la referencia secuencial y operaciones compartidas, “*physics_openmp.cpp*” contiene la región paralela. “*renderer.cpp*” concentra GLFW, GLEW y OpenGL bajo RendererSession. “*benchmark.cpp*”, visual.cpp y csv.cpp implementan medición y persistencia.

3.3 Generación, física y colisiones

La generación usa una seed explícita, una rejilla inicial sin superposición y radios adaptados a N. En cada paso, la integración suma velocidad por delta time y refleja la componente normal al tocar un borde, para reducir tunneling, el paso se divide entre 1 y 8 subpasos según velocidad máxima y radio mínimo.

Una rejilla uniforme de tamaño de celda igual al diámetro máximo reduce candidatos: cada burbuja se compara solo con su celda y ocho vecinas. Un contacto se separa en la normal y recibe un impulso elástico con restitución 0.96 y masa proporcional al área. Los pares se ordenan antes de resolver, lo que da una trayectoria determinista.

3.4 Gestión de recursos

RendererSession aplica RAII lo que hace que su destructor invoque close y destruye recursos en orden inverso. La clase no se copia ni se mueve, por lo que existe un único dueño. La campaña FPS verifica que la ventana no se minimice, redimensione o cierre. La escritura CSV valida el esquema y comprueba flush y close antes de informar éxito.

4. Paralelización con PCAM

4.1 Partitioning

El movimiento de cada burbuja es independiente durante una fase y se particiona por índices. La detección también se particiona por el índice del primer elemento, mientras cada hilo mantiene un vector local de CollisionPair. La construcción de la rejilla y la resolución modifican estructuras compartidas y no se distribuyen.

4.2 Communication

Los hilos leen la misma rejilla y posiciones. Al finalizar la detección, critical combina las listas locales en pairs. Una barrera explícita garantiza que el vector esté completo. Single ordena y resuelve los pares; su barrera implícita publica las posiciones y collisionPairsFound antes de continuar.

4.3 Agglomeration y Mapping

Una sola región parallel abarca todos los pasos del benchmark para evitar crear y destruir el equipo repetidamente. Schedule(static) asigna intervalos contiguos y reproducibles. configureOpenMpThreads desactiva el ajuste dinámico, impide solicitar más hilos que procesadores disponibles y comprueba el tamaño real del equipo.

La referencia secuencial también conserva su CollisionGrid y vector de pares durante todos los pasos. Además, ambas rutas terminan las iteraciones del solver cuando no quedan contactos. Así se evita comparar una variante que reasigna memoria o ejecuta trabajo inútil.

5. Metodología experimental

5.1 Benchmark físico

El benchmark genera el estado antes del reloj y no inicializa GLFW, GLEW u OpenGL. Se fija seed=42, delta time=1/60 s y cinco millones de actualizaciones por caso: N=1,000 con 5,000 pasos; N=10,000 con 500; N=50,000 con 100. Se evalúan Secuencial 1T y OpenMP 1T, 2T, 4T y 8T. Cada configuración tiene diez ejecuciones independientes, en orden serial, sin descartar observaciones.

El tiempo usa steady_clock y cubre exclusivamente los pasos de física. El resumen calcula media, mínimo, máximo y desviación estándar muestral. Para cada N, la media secuencial es el denominador común de speedup y eficiencia. Un checksum consume todos los campos.

5.2 Campaña FPS

La campaña visual usa N=1,000, 10,000 y 25,000; Secuencial 1T y OpenMP 4T; VSync ON; 1 s de calentamiento y 3 s de medición. Cada configuración se repite diez veces. Se registran resolución, refresco, cuadros, FPS medio, tiempo medio y p95 de cuadro, mínimo de intervalos de 0.5 s y porcentaje de intervalos bajo 60 FPS.

5.3 Entorno y reproducibilidad

Las mediciones se realizaron en Windows 11 AMD64 con 32 procesadores lógicos informados, GCC MSYS2 UCRT64 16.2.0 y Python 3.13.14. Los scripts compilan con C++17 y advertencias estrictas, producen CSV primarios, validan repeticiones y checksums, y regeneran las figuras.

6. Resultados

6.1 Tiempo, speedup y eficiencia

N	Variante	T	Media ms	Desv. ms	S(p)	E(p)
1,000	OpenMP	1	1,969.30	7.37	0.990	0.990
1,000	OpenMP	2	2,597.55	47.56	0.750	0.375
1,000	OpenMP	4	3,460.35	37.21	0.563	0.141
1,000	OpenMP	8	6,059.97	82.24	0.322	0.040
1,000	Secuencial	1	1,948.66	4.96	1.000	1.000
10,000	OpenMP	1	7,200.90	31.86	0.988	0.988
10,000	OpenMP	2	4,351.89	57.73	1.635	0.817
10,000	OpenMP	4	2,878.76	74.80	2.471	0.618
10,000	OpenMP	8	2,816.86	161.87	2.526	0.316
10,000	Secuencial	1	7,114.39	44.26	1.000	1.000
50,000	OpenMP	1	12,328.13	21.83	0.986	0.986
50,000	OpenMP	2	6,841.18	59.81	1.777	0.889
50,000	OpenMP	4	4,111.50	32.88	2.957	0.739
50,000	OpenMP	8	2,851.04	44.71	4.265	0.533
50,000	Secuencial	1	12,158.86	40.01	1.000	1.000

Tabla 1. Diez mediciones por fila; tiempo medio del kernel, speedup y eficiencia.

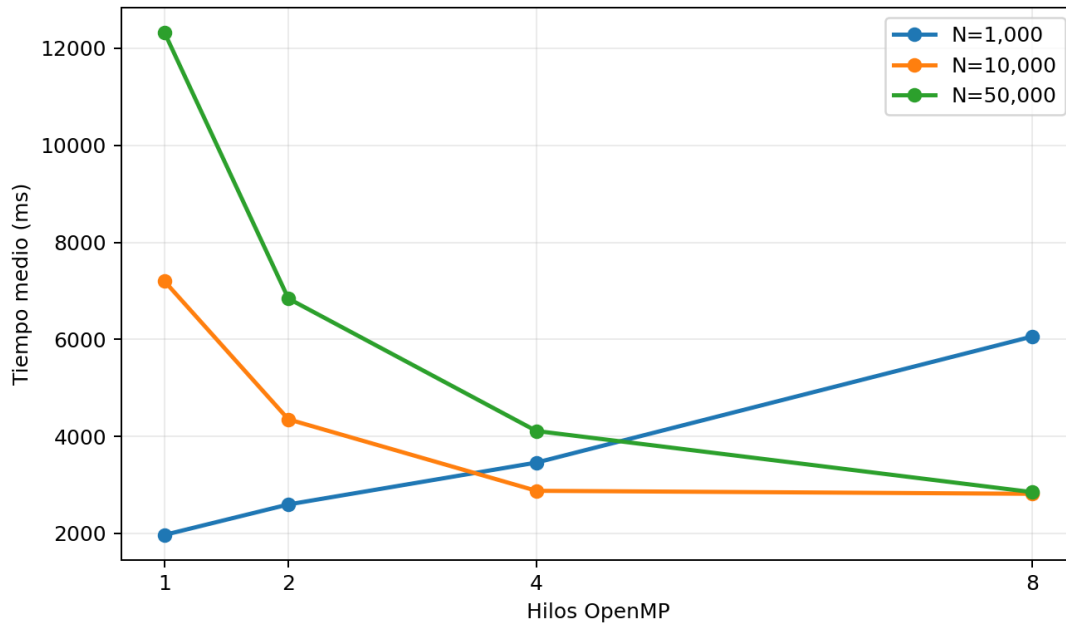


Figura 3. Tiempo medio de las configuraciones OpenMP.

6.2 Rendimiento visual

N	Variante	FPS medio	Desv.	p95 ms	Intervalos <60
1,000	OpenMP 4T	240.01	0.04	4.74	0%
1,000	Secuencial 1T	240.00	0.02	4.76	0%

10,000	OpenMP 4T	154.30	2.86	8.26	0%
10,000	Secuencial 1T	40.13	3.65	56.13	88%
25,000	OpenMP 4T	28.74	1.05	55.43	100%
25,000	Secuencial 1T	6.95	0.08	179.22	100%

Tabla 2. Diez mediciones visuales por fila con VSync ON.

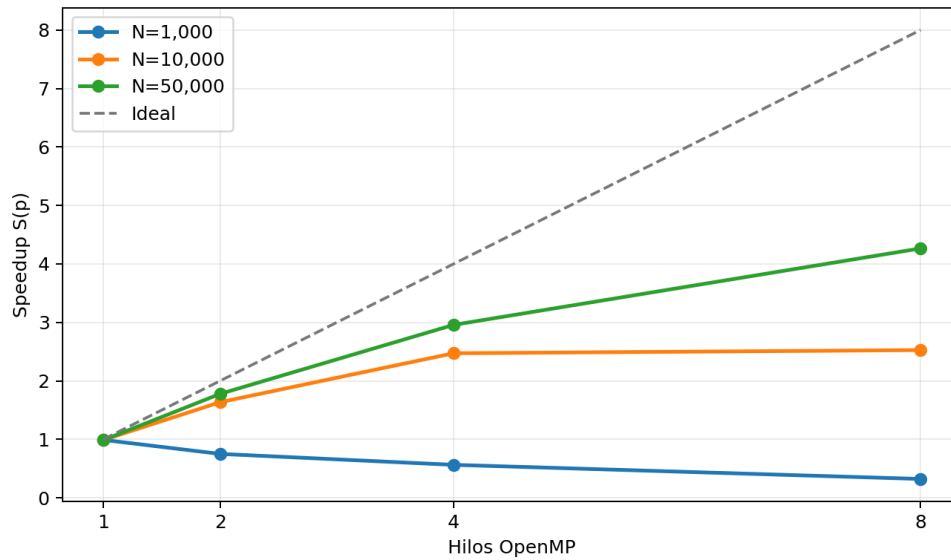


Figura 4. Speedup medido frente al ideal.

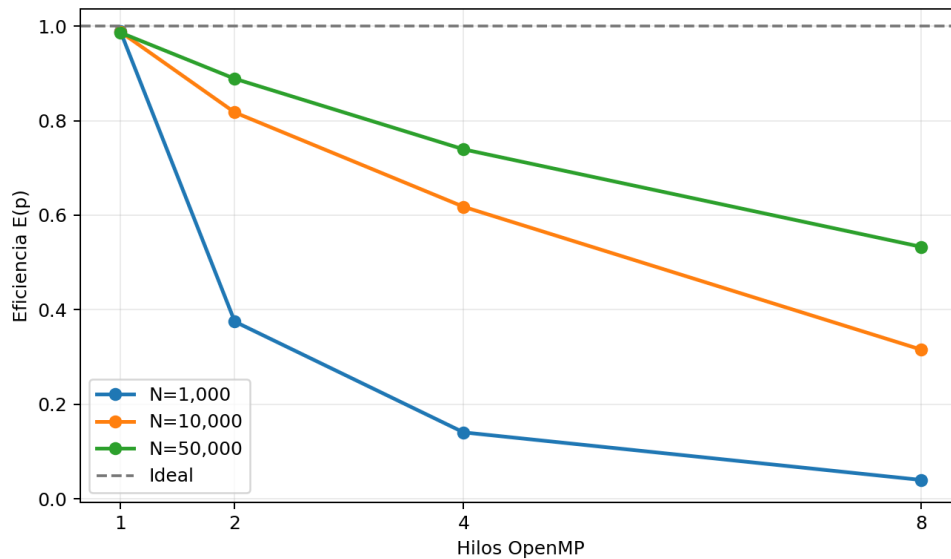


Figura 5. Eficiencia medida por cantidad de hilos.

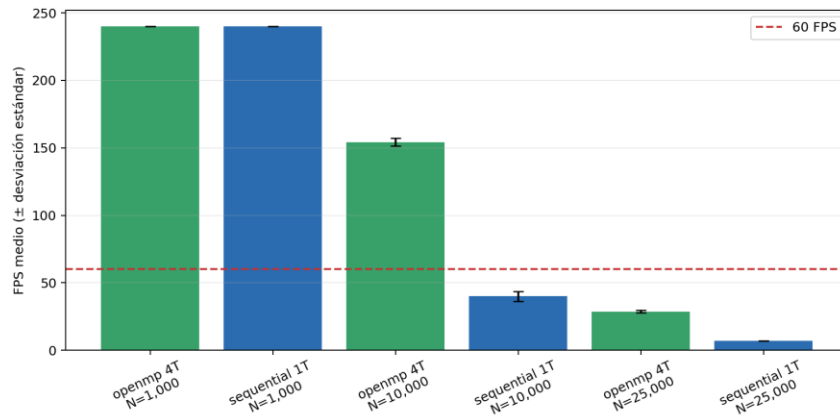


Figura 6. FPS medio y desviación estándar; línea de referencia de 60 FPS.

7. Análisis y discusión

7.1 Tamaño del problema

Con $N=1,000$, la referencia tarda 1,948.66 ms y OpenMP agrega más sincronización que trabajo útil: 8 hilos alcanzan solo 0.322x. No es una falla de corrección, sino granularidad insuficiente. Con $N=10,000$, 2 hilos ya alcanzan 1.635x y 4 hilos 2.471x. Con $N=50,000$, el mejor caso medido es 4.265x con 8 hilos.

7.2 Escalabilidad y eficiencia

OpenMP 1T queda entre 0.986x y 0.990x, lo que cuantifica el costo base de la región y sus constructos. En $N=50,000$ la eficiencia es 0.889 con 2 hilos, 0.739 con 4 y 0.533 con 8. La reducción refleja barreras, critical, trabajo serial en single y efectos de memoria. En $N=10,000$, 4 a 8 hilos apenas cambia 2.471x a 2.526x; por tanto 4 hilos ofrece mejor equilibrio para ese tamaño.

7.3 FPS y experiencia

En $N=1,000$ ambas variantes quedan cerca de 240 FPS, limitadas por VSync y el monitor; este techo impide usar la razón como medida de cómputo. En $N=10,000$ la versión secuencial promedia 40.13 FPS y cae bajo 60 en 88% de los intervalos, mientras OpenMP 4T promedia 154.30 FPS sin intervalos bajo 60. En $N=25,000$ la versión OpenMP mejora la experiencia de 6.95 a 28.74 FPS, pero no llega al objetivo de 60 FPS.

7.4 Selección práctica

El programa debería mantener la ruta secuencial para escenas pequeñas y activar OpenMP cuando N justifique el costo. Las mediciones sugieren que 4 hilos son apropiados alrededor de $N=10,000$ y que 8 hilos aportan valor en $N=50,000$. Esta política debe basarse en el equipo real.

8. Validación de corrección

La campaña produjo 150 filas y 30 grupos comparables. En cada grupo, Secuencial y OpenMP 1/2/4/8T parten del mismo checksum y terminan con el mismo checksum: 30 MATCH y cero diferencias. Las pruebas unitarias también comparan la referencia paso a paso con la ruta secuencial persistente y esta última con OpenMP.

Las pruebas de colisión verifican ausencia de penetración inicial, choque frontal, equivalencia persistente y penetración máxima paralela igual a cero. Las pruebas de CLI cubren valores válidos e inválidos, mientras que los analizadores prueban desviación muestral, speedup, eficiencia y rechazo de un checksum discordante.

9. Limitaciones

- Los resultados representan un equipo Windows específico; no generalizan a todas las CPU, GPU o runtimes OpenMP.
- La resolución de contactos permanece serial para conservar determinismo; limita la escalabilidad con muchos hilos.
- La rejilla se reconstruye en cada iteración y puede dominar escenarios densos.
- Las pruebas FPS duran 3 s por repetición y dependen de monitor, GPU, compositor y carga del sistema.
- La procedencia exacta de Fondo.jpg no quedó registrada; debe sustituirse por un recurso propio o de licencia comprobable antes de una publicación externa.
- El vínculo e historial Git se agregarán cuando el equipo indique el repositorio real; no se reconstruyó historial artificial.

10. Conclusiones

- La solución cumple el comportamiento visual, la física, el contador FPS, la validación defensiva y las variantes secuencial/OpenMP.
- Una comparación justa requiere el mismo solver, las mismas reglas de terminación y workspace persistente en ambas variantes.
- OpenMP no beneficia cargas pequeñas: $N=1,000$ se desacelera al aumentar hilos.
- La carga grande sí escala: $N=50,000$ alcanzó 4.265x y eficiencia 0.533 con 8 hilos.
- Los 30 grupos de checksums idénticos respaldan que la aceleración no cambió el resultado numérico.
- Los FPS confirman valor práctico en $N=10,000$, pero deben mantenerse separados del speedup del kernel.

11. Recomendaciones

- Seleccionar secuencial u OpenMP según N y una calibración breve del equipo.
- Investigar resolución por coloreo o acumulación de impulsos para reducir la sección single sin carreras.
- Medir regiones OpenMP con profiling para separar barreras, critical y ancho de banda de memoria.
- Repetir la campaña FPS por más tiempo y en más hardware si se desea generalizar conclusiones.
- Mantener CSV primarios, checksums y scripts junto con la entrega; no incluir ejecutables.
- Agregar al final el repositorio real y su historial, sin modificar artificialmente fechas o autores.

Referencias

1. OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. Noviembre de 2021. <https://www.openmp.org/spec-html/5.2/openmp.html>
2. Amdahl, G. M. "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities". *AFIPS Conference Proceedings*, vol. 30, 1967, pp. 483-485. <https://doi.org/10.1145/1465482.1465560>
3. Foster, I. *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Addison-Wesley, 1995. <https://www.mcs.anl.gov/~itf/dbpp/>
4. Grama, A.; Gupta, A.; Karypis, G.; Kumar, V. *Introduction to Parallel Computing*, 2a ed. Addison-Wesley, 2003.
5. Khronos Group. *OpenGL 3.3 Core Profile Specification*. 2010. <https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf>
6. GLFW. *Window guide and swap interval documentation*. https://www.glfw.org/docs/latest/window_guide.html

Anexos

Los anexos reúnen el flujo completo de ejecución, el catálogo de funciones y tipos, y la evidencia reproducible de las pruebas. Los valores numéricos provienen de los CSV conservados en results.

Anexo 1 - Diagrama de flujo

El programa no solicita datos de manera interactiva: recibe N y los demás parámetros por línea de comandos. Si falta un dato obligatorio o un valor no cumple los límites, muestra el error, imprime el uso y termina sin crear la ventana ni modificar un CSV.

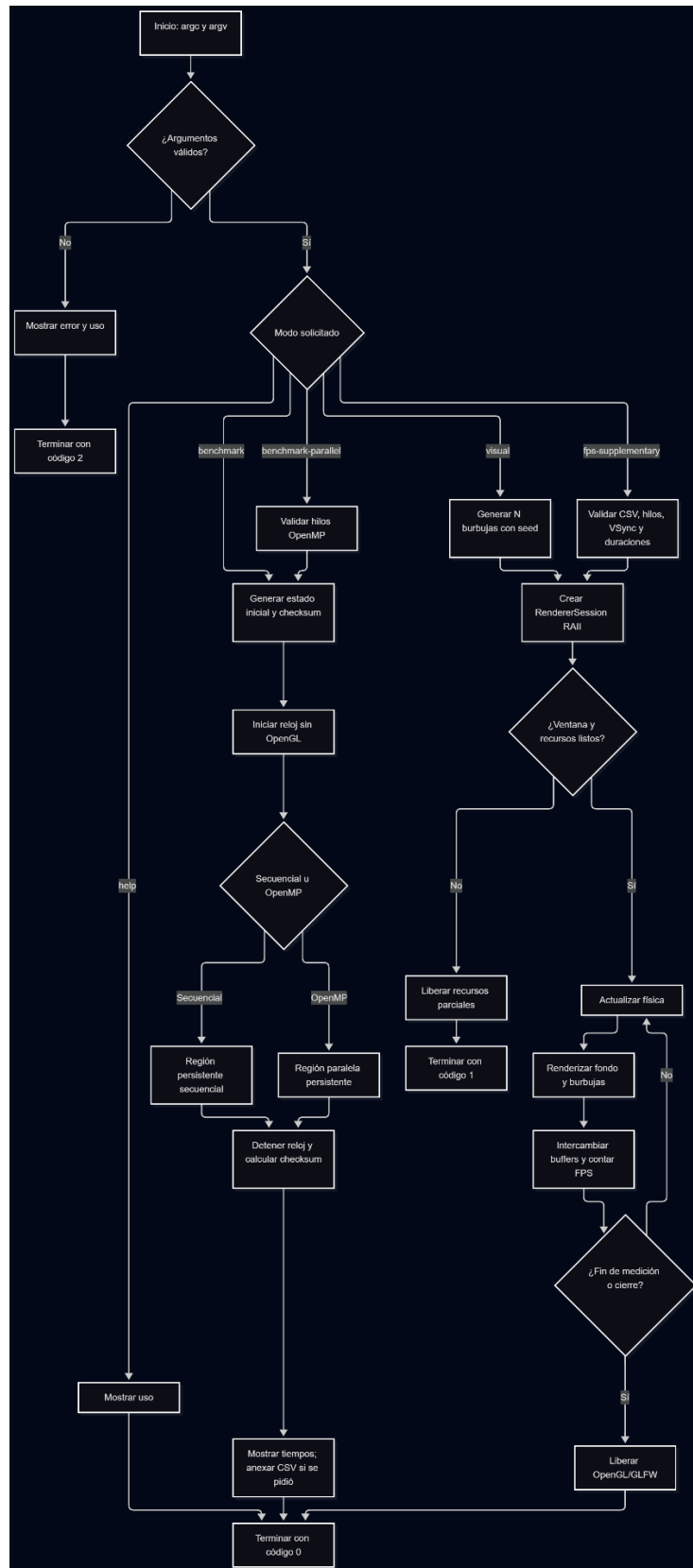


Figura A1. Flujo general del programa y sus cuatro modos de ejecución.

Paso de física y sincronización

La región crítica protege el vector compartido de pares. La barrera explícita impide que single consuma listas incompletas; la barrera implícita al terminar single hace que todos los hilos observen collisionPairsFound antes de decidir la salida. La resolución se serializa para evitar escrituras concurrentes sobre una misma burbuja y conservar un orden determinista idéntico al secuencial.

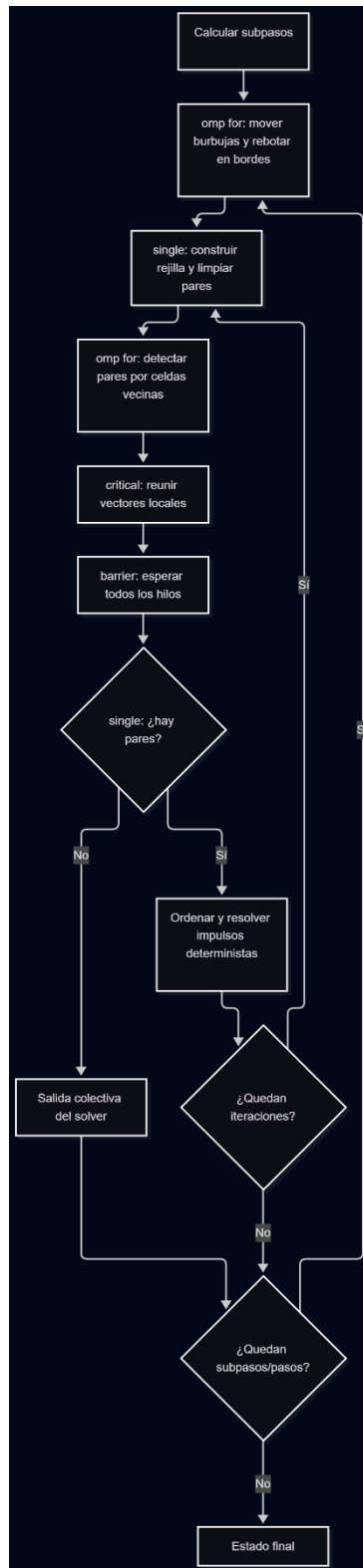


Figura A2. Paralelización y puntos de sincronización del paso de física.

Anexo 2 - Catálogo de funciones y tipos

El catálogo identifica las entradas, salidas y responsabilidades del código de autoría del equipo. Las rutinas de terceros contenidas en `stb_image.h` se excluyen.

Tipos públicos

Tipo	Campos o interfaz	Salida / responsabilidad
Vec2	float x, float y	Representa una posición o velocidad bidimensional.
Bubble	position, velocity, radius, color[4]	Estado físico y visual de una burbuja.
ProgramOptions	modo, N, seed, pasos, repetición, hilos, CSV, VSync y duraciones	Resultado validado de la línea de comandos.
BenchmarkResult	configuración, tiempos y checksums	Fila primaria del benchmark físico.
FpsResult	configuración de ventana y métricas de cuadros	Fila primaria de la prueba visual suplementaria.
RendererConfig	bubbleCount, threadCount, vsyncEnabled, fixedWindowSize	Configura la sesión gráfica.
RendererDimensions	ancho/alto lógico y del framebuffer	Permite comprobar que la ventana permanezca estable.
RendererSession	constructor, isReady(), close(), destructor	Dueño RAII no copiable de GLFW/OpenGL, libera recursos incluso ante salida temprana.

CLI y modos

Función	Entradas	Salida	Propósito
parseCommandLine	argc: int, argv: char**, options: ProgramOptions&, errorMessage: string&	bool y opciones/error por referencia	Reconoce los cuatro modos; rechaza texto, negativos, overflow, duplicados y límites inválidos.
printUsage	output: ostream&	Ninguna	Imprime sintaxis y límites.
runVisual	options: const ProgramOptions&	Código int	Ejecuta el screensaver hasta que se cierre la ventana.
runBenchmark	options: const ProgramOptions&	Código int	Mide únicamente el kernel físico, informa tiempos y opcionalmente guarda CSV.
runFpsSupplementary	options: const ProgramOptions&	Código int	Ejecuta calentamiento y medición visual controlada;

			valida estabilidad de la ventana.
main	argc: int, argv: char**	Código int	Captura excepciones, invoca la CLI y despacha ayuda, benchmark o visual.

Auxiliares privados de CLI

Función	Entradas	Salida	Propósito
parseUnsigned64	text: string_view, value: uint64_t&	bool	Convierte un decimal completo con from_chars, sin signos ni texto sobrante.
parseBubbleCount	texto, bubbleCount, error	bool	Valida el rango permitido de N.
parseSeed	texto, seed, error	bool	Valida una semilla de 64 bits sin signo.
parseSteps	texto, steps, error	bool	Valida la cantidad de pasos.
parseThreadCount	texto, hilos, error	bool	Valida la cantidad solicitada de hilos.
parseRepeat	texto, repetición, error	bool	Valida el identificador de repetición.
parseFpsPhaseSeconds	texto, permiso de cero, nombre, segundos, error	bool	Valida duraciones de calentamiento y medición.
parseVSync	texto, enabled, error	bool	Acepta únicamente on u off.
createDefaultSeed	Ninguna	uint64_t	Combina reloj y random_device para el modo visual sin seed.

Generación y persistencia

Función	Entradas	Salida	Propósito
generateBubbles	bubbleCount: size_t, seed: uint64_t, areaWidth/Height: float	vector<Bubble>	Genera posiciones no superpuestas, radios adaptados a N, velocidades y colores reproducibles.
checksumBubbleState	bubbles: const vector<Bubble>&	uint64_t	Resume todos los campos mediante FNV-1a para comprobar equivalencia.
formatChecksum	checksum: uint64_t	string	Produce hexadecimal de 16 dígitos.
appendBenchmarkCsv	ruta, BenchmarkResult, mensaje de error	bool	Verifica encabezado y agrega una fila primaria.

validateFpsCsvAppend	ruta, FpsResult, writeHeader, error	bool	Rechaza esquemas incompatibles y configuraciones repetidas.
appendFpsCsv	ruta, FpsResult, error	bool	Agrega una medición FPS y fuerza el vaciado/cierre del archivo.

Auxiliares privados de generación y CSV

Función	Entradas	Salida	Propósito
hsvToRgb	matiz, saturación y valor	array<float, 3>	Convierte el color pseudoaleatorio a RGB.
appendHashBytes	hash, valor y cantidad de bytes	Hash modificado	Incorpora una representación entera al FNV-1a.
splitSimpleCsvRow	fila sin comillas	vector<string>	Divide el esquema FPS controlado por comas.
isSameFpsConfiguration	campos y resultado	bool	Detecta una clave FPS ya existente.

Física secuencial y compartida

Función	Entradas	Salida	Propósito
updatePhysics	burbujas por referencia, deltaTime, ancho y alto	Estado modificado	Actualiza un cuadro secuencial y limita deltaTime.
updatePhysicsSequentialPersistent	burbujas, steps: uint64_t, tiempo y área	Estado modificado	Ejecuta varios pasos reutilizando rejilla y vector de pares; referencia justa del benchmark.
updateSingleBubble	una burbuja, tiempo y área	Burbuja modificada	Integra posición y refleja velocidad en bordes; kernel independiente por elemento.
calculateCollisionSubsteps	burbujas y tiempo	int	Elige 1-8 subpasos según velocidad y radio para reducir tunneling.
buildCollisionGrid	burbujas, área, CollisionGrid&	Rejilla modificada	Asigna cada burbuja a una celda uniforme.
collectCollisionPairsForBubble	estado, rejilla, índice, vector de pares	Pares agregados	Busca solapamientos en la celda propia y ocho vecinas.
resolveCollisionPairs	burbujas, pares, área	Estado modificado	Separa círculos y aplica impulso elástico con masa proporcional al área.

reflectAtBorders	posición, velocidad, radio y límite	Posición/velocidad modificadas	Reubica dentro del canvas e invierte la componente normal.
collisionPairLess	dos pares	bool	Define el orden determinista por índices.
collisionCellIndex	rejilla y posición	int	Calcula y limita la celda lineal.
resolveCollisionsSequential	burbujas, área, rejilla y pares	Estado/workspace modificados	Repite detección y resolución hasta diez veces o hasta no tener contactos.
advancePhysicsSequential	burbujas, tiempo, área y workspace	Estado modificado	Ejecuta todos los subpasos de un paso sin reasignar workspace.

CollisionGrid contiene cellSize, filas, columnas, cabezas y enlaces; evita la búsqueda completa $O(N^2)$ en escenarios dispersos. CollisionPair guarda los dos índices de un contacto y se ordena antes de resolverlo.

OpenMP

Función	Entradas	Salida	Propósito
updatePhysicsParallel	burbujas, tiempo y área	Estado modificado	Adaptador de un paso hacia la región persistente.
updatePhysicsParallelPersistent	burbujas, pasos, tiempo y área	Estado modificado	Paraleliza movimiento y detección con omp for; usa listas locales, critical, barrier y single.
configureOpenMpThreads	hilos pedidos; procesadores, hilos activos y error por referencia	bool	Impide sobresuscripción, desactiva ajuste dinámico y confirma el equipo real.
detectOpenMpThreadCount	Ninguna	int	Consulta cuántos hilos activa el runtime.

Renderizador

Función	Entradas	Salida	Propósito
render	const vector<Bubble>&	Imagen en framebuffer	Dibuja textura de fondo y círculos translúcidos.
presentRenderFrame	Ninguna	Ninguna	Intercambia buffers y procesa eventos.
rendererWindowShouldClose	Ninguna	bool	Indica cierre solicitado.
rendererTime	Ninguna	double	Devuelve reloj de GLFW.
setRendererTitle	title: const string&	Ninguna	Muestra variante, N y FPS.

rendererDimensions	Ninguna	RendererDimensions	Obtiene tamaños actuales.
queryWindowMonitorRefreshHz	Ninguna	optional<int>	Consulta refresco del monitor cuando está disponible.
measurementWindowIsStable	dimensiones esperadas y error	bool	Rechaza cierre, minimización o cambio de tamaño durante una medición.

Auxiliares privados de visualización y renderizado

Función	Entradas	Salida	Propósito
executeVisualFrame	burbujas y tiempo anterior	Estado y tiempo modificados	Obtiene delta, actualiza física, dibuja y presenta un cuadro.
glfwErrorCallback	código y descripción	Ninguna	Reporta errores de GLFW.
windowSizeCallback	ventana, ancho y alto	Estado del renderer	Actualiza dimensiones lógicas con sus mínimos.
framebufferSizeCallback	ventana, ancho y alto	Estado/Open GL	Actualiza dimensiones físicas y el viewport.
keyCallback	ventana, tecla, acción	Estado de ventana	Cierra al presionar Escape.
compileShader	tipo y fuente GLSL	GLuint	Compila un shader y reporta el log de error.
createShaderProgram	fuentes de vértice y fragmento	GLuint	Enlaza un programa y destruye shaders intermedios.
createBubbleShaderProgram	Ninguna	GLuint	Construye el programa para círculos con alpha.
createBackgroundShaderProgram	Ninguna	GLuint	Construye el programa para la textura de fondo.
createCircleGeometry	Ninguna	bool	Crea VAO/VBO del círculo unitario.
createBackgroundGeometry	Ninguna	bool	Crea VAO/VBO del cuadrilátero de pantalla.
loadBackgroundTexture	ruta de imagen	bool	Decodifica JPEG y configura la textura OpenGL.
initializeRenderImpl	RendererConfig	bool	Inicializa GLFW, ventana, GLEW, shaders, geometría y textura.

cleanupRender erImpl	Ninguna	Ninguna	Destruye recursos OpenGL, ventana y GLFW de forma idempotente.
-------------------------	---------	---------	--

Anexo 3 - Bitácora de pruebas

Ambiente

- Sistema: Windows 11, AMD64, 32 procesadores lógicos informados.
- Compilador: GCC MSYS2 UCRT64 16.2.0, C++17 y OpenMP.
- Python: 3.13.14.
- Seed: 42; paso físico: 1/60 s; ejecuciones seriales para evitar interferencia entre procesos.
- Se preservaron todas las observaciones primarias, sin eliminar outliers.

El archivo results/environment.txt conserva el ambiente registrado por el script, no una transcripción manual.

Pruebas automáticas

Comando reproducible: .\scripts\run_tests.ps1

Comprobación	Resultado
core_behavior	PASS
initial_penetration	0
head_on_collision	PASS
sequential_openmp_equivalence	PASS
persistent_equivalence	PASS
maximum_parallel_penetration	0
benchmark_fairness	PASS
Pruebas Python	3 de 3 - OK
Suite completa	PASS

La compilación secuencial y OpenMP también terminó sin advertencias con -Wall -Wextra -Wpedantic -Werror.

Benchmark físico canónico

Comandos reproducibles: py .\scripts\run_benchmark_campaign.py --overwrite y py .\scripts\analyze_benchmarks.py. Cada celda contiene 10 mediciones. El speedup usa $S(p) = \text{media_secuencial} / \text{media_paralela}(p)$ y la eficiencia $E(p) = S(p) / p$. Los tiempos corresponden al kernel físico, sin creación de ventana, renderizado ni generación inicial.

N	Pasos	Variante	Hilos	Media (ms)	Desv. (ms)	S(p)	E(p)
1,000	5,000	Secuencial	1	1,948.66	4.96	1.000	1.000
1,000	5,000	OpenMP	1	1,969.30	7.37	0.990	0.990
1,000	5,000	OpenMP	2	2,597.55	47.56	0.750	0.375
1,000	5,000	OpenMP	4	3,460.35	37.21	0.563	0.141
1,000	5,000	OpenMP	8	6,059.97	82.24	0.322	0.040
10,000	500	Secuencial	1	7,114.39	44.26	1.000	1.000

10,000	500	OpenMP	1	7,200.90	31.86	0.988	0.988
10,000	500	OpenMP	2	4,351.89	57.73	1.635	0.817
10,000	500	OpenMP	4	2,878.76	74.80	2.471	0.618
10,000	500	OpenMP	8	2,816.86	161.87	2.526	0.316
50,000	100	Secuencial	1	12,158.86	40.01	1.000	1.000
50,000	100	OpenMP	1	12,328.13	21.83	0.986	0.986
50,000	100	OpenMP	2	6,841.18	59.81	1.777	0.889
50,000	100	OpenMP	4	4,111.50	32.88	2.957	0.739
50,000	100	OpenMP	8	2,851.04	44.71	4.265	0.533

Tabla A1. Resumen del benchmark físico canónico; 10 repeticiones por configuración.

El CSV primario contiene 150 filas y 15 configuraciones. La validación contiene 30 grupos (tres tamaños por diez repeticiones), las cinco variantes de cada grupo comparten checksum inicial y final. Resultado: 30 MATCH y 0 diferencias.

Evidencia visual suplementaria

Comandos reproducibles: `py .\scripts\run_fps_campaign.py --overwrite` y `py .\scripts\analyze_fps.py`. Se obtuvieron 60 filas: $N = \{1,000; 10,000; 25,000\}$, Secuencial 1T y OpenMP 4T, 10 repeticiones, VSync ON, 1 s de calentamiento y 3 s de medición.

N	Variante	Hilos	FPS medio	Desv. FPS	p95 (ms)	Intervalos <60 FPS
1,000	Secuencial	1	240.00	0.02	4.76	0.0%
1,000	OpenMP	4	240.01	0.04	4.74	0.0%
10,000	Secuencial	1	40.13	3.65	56.13	88.0%
10,000	OpenMP	4	154.30	2.86	8.26	0.0%
25,000	Secuencial	1	6.95	0.08	179.22	100.0%
25,000	OpenMP	4	28.74	1.05	55.43	100.0%

Tabla A2. Evidencia visual suplementaria; 10 repeticiones por configuración.

```
$ C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Proyecto\Ver2\bubble_screensaver\build\BubbleScreensaver.exe --benchmark 10000 42 50 --repeat 1
Mode: benchmark
Implementation: sequential
Threads: 1
N: 10000
Seed: 42
Steps: 50
Repeat: 1
Delta time (s): 0.01666666667
Initial checksum: 0x4185A87E5FD38824
Final checksum: 0xA911F6C824839E9E
Total time (ms): 1017.839700
Average step time (ms): 20.356794
Time per element per step (ns): 2035.679400

$ C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Proyecto\Ver2\bubble_screensaver\build\BubbleScreensaverOpenMP.exe --benchmark-parallel 10000 42 50 4 --repeat 1
Mode: benchmark-parallel
Implementation: parallel
Threads: 4
N: 10000
Seed: 42
Steps: 50
Repeat: 1
Delta time (s): 0.01666666667
Initial checksum: 0x4185A87E5FD38824
Final checksum: 0xA911F6C824839E9E
Total time (ms): 410.924200
Average step time (ms): 8.218484
Time per element per step (ns): 821.848400
OpenMP strategy: persistent parallel region
OpenMP processors available: 32

CSV canónico conservado: 150 filas.
```

Figura A3. Captura de una medición secuencial y OpenMP con checksum coincidente.